

Méthodes CIM

Objectif

Manipuler l'infrastructure WMI/CIM comme une véritable API système. L'objectif est de concevoir un script d'automatisation : il doit être **idempotent** (ne rien faire si l'état est déjà correct) et s'appuyer sur une validation stricte des **codes de retour numériques** pour garantir le succès d'une pipeline CI/CD.

Exercice 1 : Découverte de l'API (Reconnaissance)

Plutôt que d'utiliser les cmdlets classiques, nous allons dialoguer directement avec le noyau Windows pour gérer les services.

1. Utilisez Get-CimClass pour interroger la classe Win32_Service.
2. Listez les méthodes disponibles pour cette classe.

Aide syntaxique :

```
(Get-CimClass -ClassName Win32_Service).CimClassMethods
```

3. Isolez la méthode ChangeStartMode et affichez les paramètres qu'elle attend en entrée (InParameters).

Question : Quel est le nom exact du paramètre permettant de modifier le type de démarrage (Automatique, Manuel, Désactivé) ?

Exercice 2 : Lecture d'État et Idempotence (Infrastructure as Code)

En automatisation CI/CD, un script doit pouvoir être lancé 100 fois sans rien casser. S'il n'y a rien à changer, il ne doit rien faire et renvoyer un code de succès.

1. Déclarez une variable \$NomService contenant la valeur "Spooler" (Spouleur d'impression).
2. Utilisez Get-CimInstance pour isoler spécifiquement ce service en appliquant un filtre WQL strict.

Aide syntaxique :

```
$ServiceTarget = Get-CimInstance -ClassName Win32_Service -Filter "Name = '$NomService'"
```

3. Mettez en place une condition logique (if / else) sur la propriété StartMode :
 - Si le mode de démarrage est **déjà** réglé sur "Auto" :
Affichez en Vert : "[IDEMPOTENCE] L'état du service est déjà conforme. Aucune action requise."
 - Sinon :
Affichez en Jaune : "[DRIFT] L'état n'est pas conforme. Modification requise."

Exercice 3 : Exécution d'Instance et Code de Retour (Le Standard Pipeline)

Vous allez maintenant modifier le mode de démarrage de ce service, non pas via la cmdlet classique, mais en invoquant l'API CIM et en contrôlant mathématiquement sa réponse.

1. Dans le bloc else de l'exercice précédent (lorsque l'état nécessite une modification), créez une table de hachage nommée \$Arguments contenant la clé StartMode avec la valeur "Automatic".
2. Envoyez votre variable \$ServiceTarget dans le pipeline vers Invoke-CimMethod pour appeler la méthode ChangeStartMode. Stockez le résultat dans une variable \$Resultat.

Aide syntaxique :

```
$Resultat = $ServiceTarget | Invoke-CimMethod -MethodName "ChangeStartMode"  
-Arguments $Arguments
```

3. Mettez en place la validation stricte de l'orchestrateur (Contrôle du ReturnValue) :
 - Si \$Resultat.ReturnValue est égal à 0 :
Affichez en Vert : "[SUCCÈS] Le mode de démarrage a été modifié. Code : 0"
 - Si le code de retour est différent de 0 :
Levez une exception bloquante (qui ferait échouer une pipeline CI/CD) avec la commande throw :
throw "[ÉCHEC FATAL] L'API Windows a refusé la modification. Code d'erreur :
\$(\$Resultat.ReturnValue)"

Test de validation : Changez manuellement le type de démarrage du Spouleur d'impression sur "Manuel" dans l'interface Windows (services.msc), puis exécutez votre script deux fois de suite pour observer l'idempotence.